Task 12: Explore a Cryptography Concept

Concept: Hashing | Rupashi Sethi

What is hashing?

A hash function turns any data into a short, fixed-size fingerprint. SHA-256 always gives 64 hex characters. The same input always gives the same output, a tiny change gives a completely different output, and you cannot turn the hash back into the original data. It exists so we can check that data has not changed, and store passwords without keeping the passwords themselves.

Demonstration (Python, synthetic data)

Demo	Result
Fingerprint	Known test values match, e.g. SHA-256 of "abc" = ba7816bf... Also matches the system sha256sum tool.
Avalanche	Changing 100 to 900 in a message flipped 114 of 256 bits (44.5%).
Integrity	An edited file no longer matched its original hash: tampering detected.
Limit 1	An attacker can edit a file and publish a new hash, so a plain hash does not prove who made it. HMAC with a secret key rejected the forged tag.
Limit 2	Two users with the same password had identical unsalted hashes, and a tiny lookup table found it. Salted PBKDF2 gave different hashes.

What hashing can and cannot protect

Can: detect changes, store passwords safely (with salt and a slow function), and with HMAC prove a message came from someone with the key.
Cannot: keep data secret (it is not encryption), prove who made a file on its own, save weak passwords from guessing, or make MD5 and SHA-1 safe, because both have known collisions.

Testing

34 automated tests pass, including official test values for SHA-2 and HMAC (RFC 4231), file hashing, command-line errors and the full demo.

Sources

NIST FIPS 180-4 (SHA-2); RFC 2104 (HMAC); OWASP Password Storage Cheat Sheet; SHAttered, the 2017 SHA-1 collision.